

# NEXTONES — Phase 3C : déploiement live + procédure de bascule

**Statut** : Phase 3C installée, mode triple verrou actif. Aucun ordre live ne peut être envoyé tant que les 3 verrous ne sont pas relâchés manuellement.

**Compte test** : ActivTrades, capital 800 EUR, MetaApi account 1004815.

## 1. État après installation

### Fichiers déployés

| Fichier                            | Rôle                                 | Marker                             |
|------------------------------------|--------------------------------------|------------------------------------|
| bridge_config.py                   | Constantes runtime (verrous)         | n/a                                |
| nextones-broker-router.py          | Décideur shadow/live/reject          | n/a (module standalone)            |
| metaapi_provider.py                | place_order réel avec garde DRY_RUN  | [NEXTONES-META-API-PLACE-ORDER-V1] |
| execution_engine.py                | Wiring routeur avant shadow_executor | [NEXTONES-BROKER-ROUTER-V1]        |
| nextones-broker-shadow-executor.py | Shadow trail (intact, Phase 3A)      | [NEXTONES-SHADOW-EXEC-V1]          |

### Schéma DB

```
-- Phase 3A + 3C
ALTER TABLE broker_shadow_orders ADD COLUMN is_live INTEGER DEFAULT 0;
CREATE INDEX idx_shadow_is_live ON broker_shadow_orders(is_live, status);

-- Phase 3B
CREATE TABLE broker_reconciliation_runs (...);
CREATE TABLE broker_reconciliation_log (...);
```

### Triple verrou de sécurité (bridge\_config.py)

```
BROKER_LIVE_ENABLED = False          # [verrou 1] Live globalement désactivé
LIVE_DRY_RUN = True                   # [verrou 2] place_order inerte même si live=True
LIVE_INSTRUMENTS = set()              # [verrou 3] Whitelist vide
MAX_LIVE_NAV = 300.0                  # Cap NAV total exposé (EUR)
MAX_LIVE_NOTIONAL_PER_ORDER = 100.0  # Cap par ordre (EUR)
BROKER_LIVE_ACCOUNT = "ACTIVTRADES"
BROKER_SHADOW_ENABLED = True          # Phase 3A toujours active
```

## 2. Architecture decision tree

```
execute_order(conn, instrument_id, side, qty, ...)
├─ risk_pretrade (V2) ─────────→ reject → return
├─ [NEXTONES-BROKER-ROUTER-V1]
│   route_order(ticker, side, qty, asset_class, entry_price, cycle_id)
│   └─ verrou 1 : not BROKER_LIVE_ENABLED → route=shadow (live_disabled)
│   └─ verrou 2 : ticker not in LIVE_INSTRUMENTS → route=shadow (not_in_live_whi
│   └─ verrou 2bis : equity + marché fermé → route=reject (market_closed)
│   └─ translator (resolver ActivTrades + lots/specs)
│       └─ rejected → route=reject (translator_reject)
│   └─ verrou 4 : notional > MAX_LIVE_NOTIONAL_PER_ORDER → route=reject
│   └─ verrou 5 : NAV + notional > MAX_LIVE_NAV → route=reject
│   └─ verrou 6 : crypto/equity market_closed → route=reject
│   └─ ALL PASS → route=live
│       └─ LIVE_DRY_RUN=True → reason=live_dry_run (audit only)
│       └─ LIVE_DRY_RUN=False → reason=live_armed + metaapi.place_order()
├─ branche reject → UPDATE orders status=rejected_router, return
├─ branche live → (si armed) place_order + continue
└─ [NEXTONES-SHADOW-EXEC-V1] shadow_executor en parallèle (INSERT broker_shadow_orc
```

## 3. Validation déploiement

Exécuter dans l'ordre, tous doivent retourner **PASS** :

```
# A. Routeur en isolation (6 scénarios)
py -3.13 nextones-validate-router.py
# Cible : 19/19 PASS

# B. Wiring runtime
py -3.13 nextones-validate-router-wired.py
# Cible : tous OK

# C. Reconciler full-stack
py -3.13 nextones-validate-broker-reconciler.py
# Cible : 35/35 PASS

# D. Market guard
py -3.13 nextones-test-market-guard-runtime.py
# Cible : SKIP correct hors heures marché, PASS sinon
```

## 4. Procédure de bascule LIVE — Step-by-step

## Phase A — Préparation (toujours en shadow)

### 1. Vérifier état actuel :

```
Get-Content .\bridge_config.py
```

Doit afficher :

- `BROKER_LIVE_ENABLED = False`
- `LIVE_DRY_RUN = True`
- `LIVE_INSTRUMENTS = set()`

### 2. Vérifier l'API tourne :

```
py -3.13 -m uvicorn api_server_with_static:app --host 0.0.0.0 --port 8000
```

### 3. Snapshot baseline :

```
py -3.13 -c "import sqlite3; c=sqlite3.connect('thesium.db'); print('shadow rows:'
```

## Phase B — Activer live en DRY\_RUN (observation 24h)

### 4. Éditer bridge\_config.py :

```
BROKER_LIVE_ENABLED = True          # ← ON
LIVE_DRY_RUN = True                  # reste True : inerte
LIVE_INSTRUMENTS = {"LINK"}          # ← whitelist crypto uniquement
MAX_LIVE_NAV = 300.0                 # inchangé
MAX_LIVE_NOTIONAL_PER_ORDER = 100.0 # inchangé
```

### 5. Restart API :

```
powershell -ExecutionPolicy Bypass -File .\nextones-kill-uvicorn.ps1
py -3.13 -m uvicorn api_server_with_static:app --host 0.0.0.0 --port 8000
```

### 6. Observer 24h logs broker\_router\_decision :

```
py -3.13 -c "
import sqlite3
c=sqlite3.connect('thesium.db')
for r in c.execute(\"SELECT created_at, payload FROM event_log WHERE event_type='
    print(r[0], r[1])
"
```

Attendu : pour chaque ordre LINK :

- `route=live reason=live_dry_run`
- `est_notional_eur < 100 EUR` (cap par ordre)
- `live_nav_eur + est_notional_eur < 300 EUR` (cap NAV)

### 7. Critères GO/NO-GO après 24h :

- Au moins 1 cycle complet sans erreur stderr [WARN] [NEXTONES-BROKER-ROUTER-V1]
- Aucune décision aberrante (volume\_lots cohérent, broker\_symbol = LINKUSD)
- nextones-validate-broker-reconciler.py retourne 35/35 PASS

## Phase C — Armement live réel

### 8. Éditer bridge\_config.py :

```
BROKER_LIVE_ENABLED = True
LIVE_DRY_RUN = False           # ← ARMED
LIVE_INSTRUMENTS = {"LINK"}
```

### 9. Restart API.

### 10. Premier ordre LIVE — surveillance immédiate :

```
# Tail event_log toutes les 30s
while ($true) {
    py -3.13 -c "
import sqlite3
c=sqlite3.connect('thesium.db')
for r in c.execute(\"SELECT created_at, event_type, payload FROM event_log WHERE
    print(r)
"
    Start-Sleep -Seconds 30
}
```

### 11. Vérification MetaApi côté broker : connecter ActivTrades, vérifier que l'ordre apparaît avec le bon symbole, volume, side.

### 12. Reconciler après premier fill :

```
py -3.13 nextones-validate-broker-reconciler.py
```

Cible : pas de drift qty/notional.

## 5. Rollback d'urgence

### Niveau 1 — Geler les ordres live (immédiat)

```
# bridge_config.py
BROKER_LIVE_ENABLED = False
```

Restart API. Tous les nouveaux ordres repassent en shadow.

## Niveau 2 — Restaurer un état pré-Phase 3C

Backups automatiques disponibles :

- bridge\_config.py.bak.20260531-104324
- metaapi\_provider.py.bak.20260531-104325
- execution\_engine.py.bak.routerwiring.{ts}

```
Copy-Item .\bridge_config.py.bak.20260531-104324 .\bridge_config.py -Force
Copy-Item .\metaapi_provider.py.bak.20260531-104325 .\metaapi_provider.py -Force
Copy-Item .\execution_engine.py.bak.routerwiring.{ts} .\execution_engine.py -Force
```

## Niveau 3 — Fermer manuellement les positions broker

Via MetaApi RPC ou ActivTrades web :

- Cancel pending orders
- Close open positions sur la whitelist

## 6. Monitoring & alertes

### Requêtes utiles

Décisions routeur des dernières 24h :

```
SELECT
  json_extract(payload, '$.route') as route,
  json_extract(payload, '$.reason') as reason,
  COUNT(*) as n
FROM event_log
WHERE event_type = 'broker_router_decision'
  AND created_at > datetime('now', '-1 day')
GROUP BY route, reason
ORDER BY n DESC;
```

Exposition live actuelle :

```
SELECT
  thesium_ticker,
  side,
  SUM(est_notional) as notional_total,
  COUNT(*) as n_orders
FROM broker_shadow_orders
WHERE is_live = 1
  AND status IN ('open', 'filled')
GROUP BY thesium_ticker, side;
```

Place\_order errors :

```
SELECT created_at, payload
FROM event_log
WHERE event_type = 'broker_place_order'
      AND json_extract(payload, '$.accepted') = 0
ORDER BY id DESC
LIMIT 20;
```

### Alertes recommandées (à implémenter dans Phase 4)

- NAV utilisé > 80% de MAX\_LIVE\_NAV → alerte
- Plus de 5 rejets router consécutifs → alerte
- place\_order failure rate > 10% sur 1h → alerte critique

## 7. Limites & dette technique connue

- **Pas de gestion SL/TP au niveau routeur** : market order seul. Les SL/TP devront être ajoutés Phase 3D si nécessaires côté exécution.
- **Pas de re-routing après rejection** : un ordre route=reject reste rejeté (pas de retry shadow auto). Comportement strict Règle A.
- **Conversion USD → EUR fixée à 1.08** (EURUSD\_FALLBACK) — accepter en attendant un flux temps réel.
- **Colonne is\_live non encore utilisée par shadow\_executor** : actuellement seul le routeur la touche. Phase 3D : flag automatique côté shadow\_executor quand route=live.

## 8. Phase 4 — pré-requis

Avant d'attaquer Phase 4 (autonomie ordres + multi-instruments) :

- 1 semaine en LIVE sur LINK sans incident
- Reconciler 7/7 jours PASS
- NAV utilisé stable, pas de drift inattendu
- Couverture multi-symbols : ajouter ETH, BTC, SOL à LIVE\_INSTRUMENTS un par un avec 48h d'observation chacun

*Document généré 2026-05-31 — révision après chaque promotion de phase.*